07.10.2026

TOMs

Einleitung

Die DSGVO verpflichtet Unternehmen, durch geeignete technische und organisatorische Maßnahmen (die s. g. TOMs) ein dem Risiko angemessenes Schutzniveau für personenbezogene Daten sicherzustellen.

Technische Maßnahmen

Technische Maßnahmen sind bspw.

- Verschlüsselung
- Zugriffskontrollen
- Firewalls
- Pseudonymisierung
- Regelmäßige Datensicherungen
- uvm.

Organisatorische Maßnahmen

Organisatorische Maßnahmen sind…

- Mitarbeiterschulungen
- Berechtigungskonzepte nach dem Least-Privilege-Prinzip
- Vertraulichkeitsvereinbarungen
- Definierte Prozesse für den Umgang mit Datenpannen
- uvm.

Stand der Technik

TOMs sind keine einmalige Maßnahme, sondern müssen regelmäßig überprüft und an neue Risiken angepasst werden.

Bei der Auswahl der TOMs gilt der Stand der Technik als Maßstab: veraltete Sicherheitsmaßnahmen, die durch bessere ersetzt werden könnten, sind nicht ausreichend.

Datenschutzbeauftragter

Einleitung

Bestimmte Unternehmen sind nach der DSGVO verpflichtet, einen Datenschutzbeauftragten (DSB) zu benennen.

Die Pflicht besteht für öffentliche Stellen grundsätzlich immer, für private Unternehmen wenn in der Regel mindestens 20 Personen ständig mit der automatisierten Verarbeitung personenbezogener Daten beschäftigt sind, oder wenn besonders sensible Daten in großem Umfang verarbeitet werden.

Aufgaben

- Überwacht die Einhaltung der Datenschutzvorschriften im Unternehmen
- Berät die Unternehmensleitung und die Mitarbeiter
- Ist Ansprechpartner für Betroffene und Aufsichtsbehörden
- Führt das Verzeichnis der Verarbeitungstätigkeiten.

Der DSB muss seine Aufgaben unabhängig wahrnehmen können und genießt daher besonderen Kündigungsschutz und darf wegen seiner Tätigkeit nicht benachteiligt werden.

Datenpannen

Meldepflicht bei Datenpannen

Eine Datenpanne (im DSGVO-Sprachgebrauch „Verletzung des Schutzes personenbezogener Daten“) liegt vor, wenn personenbezogene Daten versehentlich oder unrechtmäßig vernichtet, verloren, verändert oder unbefugt offengelegt werden.

Typische Beispiele sind: ein gestohlenes unverschlüsseltes Notebook mit Kundendaten, ein Ransomware-Angriff auf Personaldaten, oder eine versehentlich öffentlich zugängliche Datenbank.

Meldepflicht bei Datenpannen

Bei einer Datenpanne, die voraussichtlich ein Risiko für die Rechte und Freiheiten natürlicher Personen darstellt, muss das Unternehmen die zuständige Aufsichtsbehörde innerhalb von 72 Stunden nach Bekanntwerden informieren.

Ist das Risiko für Betroffene hoch, müssen auch diese unverzüglich benachrichtigt werden.

Unternehmen sollten daher klare interne Prozesse für die Erkennung, Dokumentation und Meldung von Datenpannen etabliert haben.

Prüfungsrelevante Einordnung



In der AP2 wird Datenschutz meist als Fallszenario oder Zuordnungsaufgabe geprüft.

Eine typische Aufgabe könnte lauten:
„Ein Unternehmen möchte die E-Mail-Adressen seiner Kunden für einen Newsletter nutzen. Die Adressen wurden ursprünglich nur für die Auftragsabwicklung erhoben. Ist das zulässig, und wenn nein, was muss das Unternehmen tun?“

Prüfungsrelevante Einordnung



Die erwartete Antwort erklärt, dass die Nutzung für den Newsletter gegen das Zweckbindungsprinzip verstößt: die Daten wurden für einen anderen Zweck erhoben.

Für den Newsletter braucht das Unternehmen eine eigene Rechtsgrundlage, typischerweise eine ausdrückliche Einwilligung der Kunden.

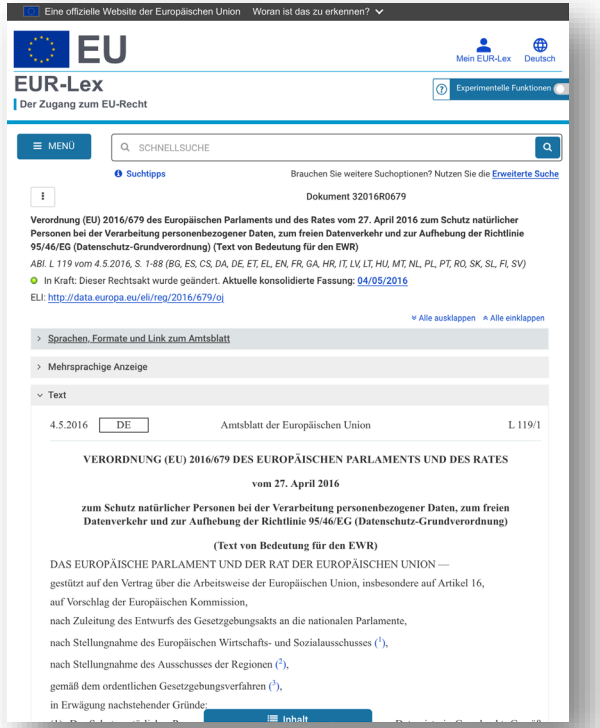
Wer zusätzlich die 72-Stunden-Meldepflicht bei Datenpannen und die Rolle des Datenschutzbeauftragten korrekt einordnet, zeigt vollständiges Verständnis der DSGVO-Grundlagen.

Aufgabe

Wenn Sie das bisher nicht gemacht haben…

- Schauen Sie mal rein.

🌐 https://eur-lex.europa.eu/eli/reg/2016/679/oj?locale=de



Standardvertragsklauseln

Standardvertragsklauseln (SCC, „Standard Contractual Clauses“) sind von der EU-Kommission vorformulierte Vertragsbausteine, mit denen Unternehmen personenbezogene Daten rechtmäßig in Länder außerhalb des Europäischen Wirtschaftsraums übermitteln können. Rechtsgrundlage ist Art. 46 Abs. 2 lit. c DSGVO.

- Verantwortlicher an Verantwortlichen
- Verantwortlicher an Auftragsverarbeiter
- Auftragsverarbeiter an Auftragsverarbeiter
- Auftragsverarbeiter an Verantwortlichen

Themenblock 11
Programmieren/Pseudocode

Einleitung

Fachinformatiker Systemintegration sind keine Softwareentwickler, aber sie arbeiten täglich mit Skripten, Automatisierungen und konfigurierbaren Systemen.

Ob Backup-Skript, Monitoring-Auswertung oder Deployment-Automatisierung: Grundlegende Programmierkenntnisse sind in der FISI-Praxis unverzichtbar.

In der AP2 wird kein vollständiges Programm erwartet, sondern eher die Fähigkeit, einfache Algorithmen in Pseudocode lesen, verstehen und selbst formulieren können.

Pseudocode ist keine echte Programmiersprache, sondern eine strukturierte und sprachunabhängige Beschreibung eines Algorithmus.

Was ist Programmieren?

- Programmieren heißt, einem Computer eine eindeutige Schritt-für-Schritt-Anweisung zu geben.
- Denn ein Computer denkt nicht mit, sondern tut exakt das, was man ihm sagt.

Daher ist unsere Kernaufgabe (als Programmierer): ein Problem in ausführbare Einzelschritte zerlegen.



Übersicht

- Algorithmus
- Kontrollstrukturen
- Variablen, Datentypen und Operatoren
- Arrays und Listen
- Funktionen und Prozeduren
- Pseudocode
- Einfache Algorithmen: Suchen und Sortieren

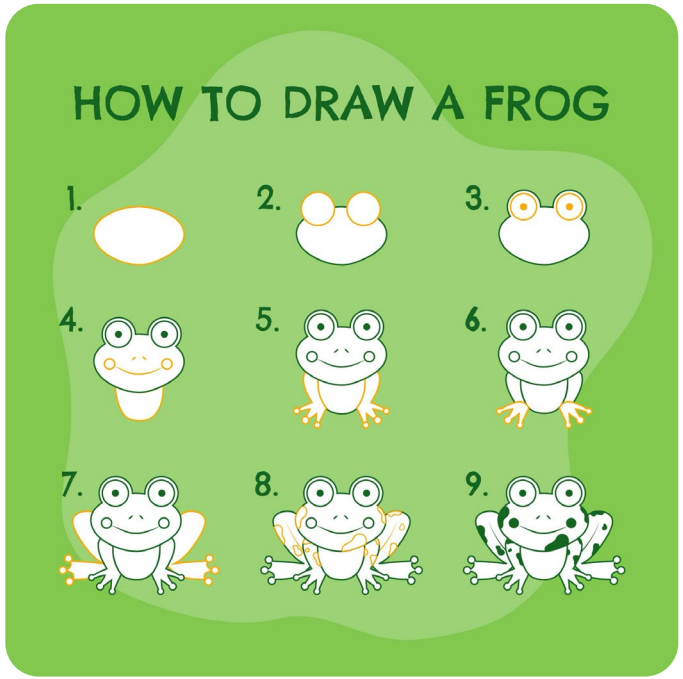
Algorithmus

Was ist ein Algorithmus?

Ein Algorithmus ist eine eindeutige Handlungsvorschrift zur Lösung eines Problems.

Beispiele dem Alltag:

- Kochrezept
- Wegbeschreibung
- Bauanleitung



Erst der Plan, dann der Code.

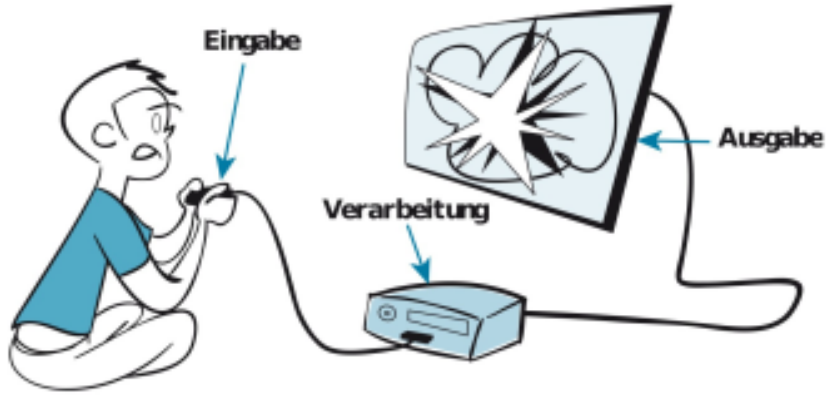
Eigenschaften eines Algorithmus

- **Eindeutig:** Jeder Schritt ist klar bestimmt.
- **Ausführbar:** Jeder Schritt ist tatsächlich durchführbar.
- **Endlich:** Endet nach endlich vielen Schritten.
- **Determiniert:** Gleiche Eingabe führt zum gleichen Ergebnis.
- **Allgemeingültig:** Löst eine ganze Problemklasse, nicht nur einen Fall.

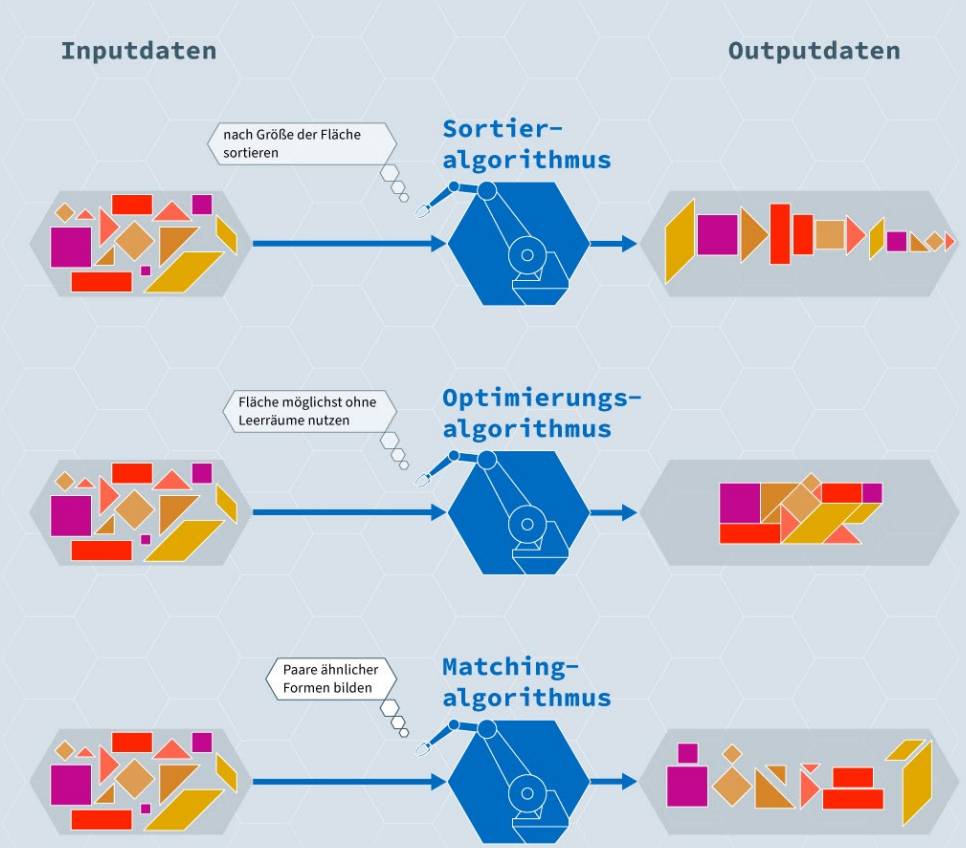
Das EVA-Prinzip

Nahezu jedes Programm folgt einem Muster:

- E – Eingabe: Daten kommen ins System (Tastatur, Datei, Sensor).
- V – Verarbeitung: Die Daten werden berechnet oder umgeformt.
- A – Ausgabe: Das Ergebnis verlässt das System (Bildschirm, Datei).



Das EVA-Prinzip



Vom Algorithmus zum Programm

- Ein Algorithmus ist die Idee bzw. der Plan, um ein Problem zu lösen (sprachneutral).
- Ein Programm ist ein Plan, der in einer Programmiersprache aufgeschrieben wurde.
- Eine Programmiersprache ist das Werkzeug, um ein Programm aufzuschreiben.

Merke: Ein Algorithmus lässt sich in vielen Sprachen (z. B. C#, Java, Python) umsetzen.

Kontrollstrukturen

Die drei Kontrollstrukturen

- Sequenz: Anweisungen nacheinander.
- Verzweigung: Entscheidung zwischen Wegen.
- Schleife/Iteration: Wiederholung von Anweisungen.

Jedes Programm besteht nur aus diesen drei Bausteinen.

Sequenz

Einfach der Reihe nach…

- Anweisungen werden einfach nacheinander, von oben nach unten, ausgeführt.
- Jede Zeile wird genau einmal ausgeführt, in der angegebenen Reihenfolge.

Sequenz

Ein Beispiel in Pseudocode:

- Eingabe einlesen
- Wert berechnen
- Ergebnis ausgeben

Anweisungen, die nacheinander ablaufen, sind die Grundlage jedes Programms.

Verzweigung – if/else

Entscheidungen treffen…

- Die Selektion ermöglicht es einem Programm, abhängig von einer Bedingung unterschiedliche Anweisungen auszuführen.
- Die if/else-Struktur ist die häufigste Form: Ist die Bedingung wahr, wird der if-Zweig ausgeführt, andernfalls der else-Zweig.

Verzweigung – if/else

In Pseudocode sieht das etwa so aus:

- WENN Temperatur > 100
- DANN ausgabe(“Überhitzt”)
- SONST ausgabe(“Normal”)
- ENDE

Selektionen können auch verschachtelt werden (also ein if innerhalb eines anderen if)oder durch SONST WENN (else if) um weitere Bedingungen erweitert werden.

Verzweigung – switch/case

Wenn es mehrere Fälle gibt…

- Wenn eine Variable gegen viele mögliche Werte geprüft werden soll, ist switch/case übersichtlicher als viele verschachtelte if/else-Konstrukte.
- Statt für jeden möglichen Wert eine eigene if-Bedingung zu schreiben, werden alle Fälle in einem switch-Block aufgelistet.

Verzweigung – switch/case

In Pseudocode:

- WÄHLE Tag AUS:
- FALL 1: ausgabe(“Montag”)
- FALL 2: ausgabe(“Dienstag”)
- STANDARD: ausgabe(“Unbekannt”)
- ENDE

Der STANDARD-Fall (default) wird ausgeführt, wenn keiner der explizit genannten Fälle zutrifft. Er ist optional, aber eine gute Praxis. Switch/case macht den Code lesbarer und ist besonders bei der Verarbeitung von Menüauswahlen oder Statuscodes nützlich.

Iteration – for

Wenn man weiß wie oft…

- Eine for-Schleife wird eingesetzt, wenn die Anzahl der Wiederholungen im Voraus bekannt ist.

Sie besteht aus drei Teilen:

- Initialisierung (Startwert des Zählers)
- Bedingung (Schleife läuft solange die Bedingung wahr ist)
- Inkrement (Zähler wird nach jeder Iteration verändert)

Iteration – for

In Pseudocode:

- FÜR i VON 1 BIS 10
- SCHRITT 1: ausgabe(i)
- ENDE

For-Schleifen werden häufig für die Verarbeitung von Arrays eingesetzt, etwa um alle Elemente eines Arrays nacheinander zu durchlaufen.

Achten Sie auf die korrekte Formulierung der Schleifenbedingung und des Inkrements.

Iteration – while-do

Wenn die Bedingung vor einem Durchlauf geprüft werden soll…

- Die while-de-Schleife wird eingesetzt, wenn die Anzahl der Wiederholungen nicht im Voraus bekannt ist.
- Die Schleife läuft solange eine Bedingung erfüllt ist.

Das nennt man kopfgesteuerte Schleife.



Iteration – while-do

In Pseudocode:

- SOLANGE Eingabe != “Ende”: Eingabe einlesen, verarbeiten
- ENDE

Wichtig: Die Bedingung wird vor jedem Durchlauf geprüft. Ist sie beim ersten Mal bereits falsch, wird die Schleife gar nicht ausgeführt.



Iteration – while-do

Kopfgesteuerte Schleife (SOLANGE Bedingung TU WAS)

- Die Bedingung wird *zuerst* geprüft.
- Der Rumpf kann also auch 0-mal laufen.
- Beispiel: Ballwechsel im Tennis.

```
SOLANGE Ball im Spiel TUE: schlage Ball zurück
```

- Bevor man schlägt, prüft man: ist der Ball überhaupt noch im Spiel?
- Landet der Aufschlag schon im Netz, schlägt man gar nicht (also 0-mal).
- Das „zuerst prüfen" ist das Kennzeichen.



Iteration – do-while

Wenn die Bedingung am Ende eines Durchlaufs geprüft werden soll…

- Die do-while-Schleife ist eine Variante, bei der die Bedingung erst nach dem ersten Durchlauf geprüft wird.
- Die Schleife wird also mindestens einmal ausgeführt.

Das nennt man fußgesteuerte Schleife.



Iteration – do-while

In Pseudocode:

- TUE: Eingabe einlesen
- SOLANGE Eingabe ungültig.

Wichtig: Die Bedingung wird nach jedem Durchlauf geprüft. Die Schleife wird also mindestens einmal ausgeführt.

Das ist nützlich für bspw. Eingabevalidierungen, bei denen mindestens ein Versuch stattfinden soll.



Iteration – do-while

Fußgesteuerte Schleife (WIEDERHOLE … SOLANGE)

- Es wird zuerst ausgeführt (erster Durchlauf der Schleife).
- Dann wird geprüft, ob ein weiterer Durchlauf notwendig ist.
- Beispiel: Golf.



```
WIEDERHOLE: schlage den Ball SOLANGE Ball nicht im Loch
```

- Man schlägt immer mindestens einmal, *dann* wird geprüft.
- Man kann ein Loch nicht mit null Schlägen beenden (deshalb „fußgesteuert“).

Wann welche Schleifenart?

Der entscheidende Unterschied zwischen den beiden „Bedingungs”-Schleifen:

- Tennis (kopfgesteuert): erst prüfen, dann handeln → evtl. 0×
- Golf (fußgesteuert): erst handeln, dann prüfen → mind. 1×



Endlosschleifen vermeiden

- Die Bedingung muss irgendwann falsch werden können.
- Bei SOLANGE / WIEDERHOLE den Zähler oder Zustand ändern.
- Sonst läuft das Programm endlos weiter.

Variablen, Datentypen und Operatoren

Variablen und Datentypen

- Eine Variable ist ein benannter Speicherplatz, der einen Wert aufnimmt und im Programmverlauf verändert werden kann.
- Jede Variable hat einen Datentyp, der bestimmt, welche Art von Wert sie speichern kann.



Variablen und Datentypen

Die wichtigsten Datentypen sind:

- Integer (ganze Zahlen)
- Float, Double oder Real (Dezimalzahlen)
- String (Zeichenketten)
- Char (einzelnes Zeichen)
- Boolean (wahr oder falsch)

Die Bezeichnungen können sich je nach Programmiersprache unterscheiden.

Operatoren

Operatoren verknüpfen Werte und Variablen:

- Arithmetische Operatoren (+ - * / mod) für Berechnungen
- Vergleichsoperatoren (= != < > <= >=) für Bedingungen
- Logische Operatoren (UND, ODER, NICHT) für die Verknüpfung von Bedingungen

In der Prüfung ist besonders die korrekte Verwendung von Vergleichs- und logischen Operatoren in Bedingungen relevant

Arrays

Was ist ein Array?

Ein Array ist eine geordnete Sammlung von Werten desselben Datentyps, die unter einem gemeinsamen Namen gespeichert werden. Dabei wird jedes Element über seinen Index angesprochen.



Achtung: der erste Index in den meisten Sprachen 0 und nicht 1.



Beispiel

In Pseudocode:

- zahlen[0] = 10
- zahlen[1] = 20
- zahlen[2] = 30

Arrays sind besonders nützlich in Kombination mit Schleifen: Eine for-Schleife kann alle Elemente eines Arrays nacheinander verarbeiten, indem der Schleifenzähler als Index verwendet wird.

Prüfungsrelevante Einordnung



In der Prüfung werden Arrays häufig für einfache Berechnungen eingesetzt: etwa die Summe aller Elemente berechnen oder den größten Wert finden.

Funktionen und Prozeduren

Einleitung

Funktionen und Prozeduren sind benannte, wiederverwendbare Codeblöcke, die eine bestimmte Aufgabe erfüllen.

- Eine Prozedur führt Anweisungen aus, gibt aber keinen Wert zurück.
- Eine Funktion führt Anweisungen aus und gibt einen Wert zurück.

Beide können Parameter entgegennehmen: also Eingabewerte, die beim Aufruf übergeben werden.

Beispiel

In Pseudocode:

- FUNKTION berechneMaximum(a, b):
- WENN a > b
- DANN RÜCKGABE a
- SONST RÜCKGABE b
- ENDE.

Der Aufruf erfolgt dann etwa so: ergebnis = berechneMaximum(10, 20)

Merksatz

Funktionen machen Code übersichtlicher, wiederverwendbar und leichter wartbar.

Statt denselben Code mehrfach zu schreiben, wird er einmal als Funktion definiert und beliebig oft aufgerufen.

Pseudocode

Was ist Pseudocode?

Pseudocode ist keine festgelegte Sprache

Es gibt keine einheitliche Syntax, die strikt eingehalten werden muss.

- Pseudocode ist eine halbformale Beschreibung eines Algorithmus.
- Liegt zwischen Umgangssprache und echtem Code.
- Keine feste Syntax.
- Lesbarkeit und Eindeutigkeit zählen hier mehr als Syntax.

Konventionen

Best Practice (Konventionen, keine Regeln)

- Schlüsselwörter großschreiben (`WENN`, `SOLANGE`, `FÜR`)
- Ein Schritt pro Zeile
- Blöcke einrücken
- Sprechende Namen verwenden (`summe` statt `s`)



Man kann (nach eigener Vorliebe) deutsche oder englische Bezeichnungen nutzen.

Prüfungsrelevante Einordnung



In der AP2 werden Pseudocode-Aufgaben in zwei Richtungen gestellt:

- Entweder wird ein gegebener Pseudocode gelesen und erklärt: was tut dieser Algorithmus, was ist das Ergebnis für eine bestimmte Eingabe?
- Oder es wird ein Problem beschrieben, und der Prüfling soll einen Pseudocode formulieren, der das Problem löst.

In beiden Fällen ist die Fähigkeit, den Ablauf Schritt für Schritt zu verfolgen, entscheidend.

Lineare Suche: Einfaches Durchsuchen

Einleitung

Die lineare Suche ist der einfachste Suchalgorithmus.

Ein Array wird von Anfang bis Ende Element für Element durchsucht, bis der gesuchte Wert gefunden wird oder das Ende des Arrays erreicht ist

Beispiel

In Pseudocode:

- FÜR i VON 0 BIS länge(array)-1:
- WENN array[i] = gesuchterWert
- DANN RÜCKGABE i
- ENDE
- RÜCKGABE -1 (nicht gefunden)

C# Python JavaScript

```
static int LineareSuche(int[] array, int gesuchterWert)
{
    for (int i = 0; i < array.Length; i++)
    {
        if (array[i] == gesuchterWert)
        {
            return i;
        }
    }
    return -1; // nicht gefunden
}
```

Nein...
Sie müssen keine konkrete Programmiersprache können für die AP2.



Beispiel

In Pseudocode:

- FÜR i VON 0 BIS länge(array)-1:
- WENN array[i] = gesuchterWert
- DANN RÜCKGABE i
- ENDE
- RÜCKGABE -1 (nicht gefunden)

C# Python JavaScript

```
def lineare_suche(array, gesuchter_wert):
    for i in range(len(array)):
        if array[i] == gesuchter_wert:
            return i
    return -1  # nicht gefunden
```

Nein...
Sie müssen keine konkrete Programmiersprache können für die AP2.



Beispiel

In Pseudocode:

- FÜR i VON 0 BIS länge(array)-1:
- WENN array[i] = gesuchterWert
- DANN RÜCKGABE i
- ENDE
- RÜCKGABE -1 (nicht gefunden)

C# Python JavaScript

```
function liniareSuche(array, gesuchterWert) {
    for (let i = 0; i < array.length; i++) {
        if (array[i] === gesuchterWert) {
            return i;
        }
    }
    return -1; // nicht gefunden
}
```

Nein...
Sie müssen keine konkrete Programmiersprache können für die AP2.



Analyse

- Die lineare Suche funktioniert auf unsortierten Arrays und ist einfach zu implementieren.
- Der Nachteil ist die Effizienz: Im schlechtesten Fall muss das gesamte Array durchsucht werden, denn bei großen Datenmengen ist das langsam.

Bubble Sort: Einfaches Sortieren

Einleitung

Bubble Sort ist ein einfacher Sortieralgorithmus, der ein Array durch wiederholtes Vergleichen und Tauschen benachbarter Elemente sortiert.

- In jedem Durchlauf werden benachbarte Elemente verglichen.
- Sind sie in der falschen Reihenfolge, werden sie getauscht.
- Nach jedem vollständigen Durchlauf ist das größte noch unsortierte Element an seiner endgültigen Position.
- Das wird so lange wiederholt, bis keine Tauschvorgänge mehr nötig sind.

Beispiel

In Pseudocode:

- FÜR i VON 0 BIS länge(array)-2:
- FÜR j VON 0 BIS länge(array)-2-i:
- WENN array[j] > array[j+1]
- DANN tausche array[j] und array[j+1]
- ENDE
- ENDE

C# Python JavaScript

```
static void BubbleSort(int[] array)
{
    for (int i = 0; i < array.Length - 1; i++)
    {
        for (int j = 0; j < array.Length - 1 - i; j++)
        {
            if (array[j] > array[j + 1])
            {
                int temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
            }
        }
    }
}
```

Nein...
Sie müssen keine konkrete Programmiersprache können für die AP2.



Beispiel

In Pseudocode:

- FÜR i VON 0 BIS länge(array)-2:
- FÜR j VON 0 BIS länge(array)-2-i:
- WENN array[j] > array[j+1]
- DANN tausche array[j] und array[j+1]
- ENDE
- ENDE

C# Python JavaScript

```
def bubble_sort(array):
    n = len(array)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if array[j] > array[j + 1]:
                array[j], array[j + 1] = array[j + 1], array[j]
```

Nein...
Sie müssen keine konkrete Programmiersprache können für die AP2.



Beispiel

In Pseudocode:

- FÜR i VON 0 BIS länge(array)-2:
- FÜR j VON 0 BIS länge(array)-2-i:
- WENN array[j] > array[j+1]
- DANN tausche array[j] und array[j+1]
- ENDE
- ENDE

C# Python JavaScript

```
function bubbleSort(array) {
    for (let i = 0; i < array.length - 1; i++) {
        for (let j = 0; j < array.length - 1 - i; j++) {
            if (array[j] > array[j + 1]) {
                let temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
            }
        }
    }
}
```

Nein...
Sie müssen keine konkrete Programmiersprache können für die AP2.



Analyse

- Bubble Sort ist nicht effizient für große Datenmengen, aber einfach zu verstehen und zu.

Info: In der AP2 wird selten erwartet, den vollständigen Bubble-Sort-Algorithmus aus dem Kopf aufzuschreiben, aber das Grundprinzip des paarweisen Vergleichens und Tauschens muss erklärt werden können.

07.10.2026

Ende